MinAI

A single-file C++ transformer, built to be read. Every mechanism of a modern LLM — attention, softmax, backprop, residual connections, causal masking — in ~780 lines of heavily commented code. Learns to reverse a list of 8 digits (and sort, shift, or mod-sum), demonstrating in miniature every idea that powers GPT-4.

Descends from Damien Boureille's ATTN-11 (PDP-11 MACRO-11 assembly) via Dave Plummer's 2.11BSD port. Portable C++17 — builds on macOS, Linux, and Windows.

Why this exists

It's easy to lose the plot on what a large language model does when every explanation starts at "96 transformer blocks". This program is the plot — the irreducible version that still contains every mechanism. If you can read ~780 lines of commented C++, you understand the skeleton of every frontier model. The only difference between this and GPT-4 is quantity: more layers, more heads, more width, more data. Not one new idea.

Quick start — two paths

If you just want to run it (no compiler, no programming required)

Prebuilt binaries for common platforms live in `bin/`. Grab the one for your platform and run it from a terminal:

Platform	File	How to run
macOS (Apple Silicon or Intel)	`bin/macos-universal/minai`	`./minai` from a terminal
Linux (x86_64)	`bin/linux-x86_64/minai`	`./minai` from a terminal
Windows (x86_64)	`bin/windows-x86_64/minai.exe`	double-click or run from `cmd` /PowerShell

See `bin/README.md` for details (including the macOS Gatekeeper first-run workaround). After it runs, `./minai --help` shows every flag.

If you want to read and modify the source

macOS / Linux (any C++17 compiler — clang++ or g++):

```
make              # builds ./minai with your default C++ compiler
./minai           # trains the default config and runs a demo
./minai --help    # see all flags
```

Windows (and an all-platforms fallback) via CMake:

```
cmake -B build
cmake --build build --config Release
./build/minai --help          # macOS/Linux
./build/Release/minai.exe     # Windows MSVC
```

No external dependencies — everything needed is in the standard library. Colored sparkline output uses Unicode block characters; works in macOS Terminal, Linux terminals, and Windows Terminal (the default on Windows 11).

Abridged output:

```
MinAI — fixed example, task=reverse, seq_len=8, batch=1, blocks=1, ffn=on, causal=off
Parameters: 2240    Training steps: 800

step    loss     correct/8
-----   ------   ----------
    1   2.2802   0/8
  ...
  800   0.0078   8/8

Loss curve (800 steps, 60 columns, higher bars = higher loss):
   2.280 |▅▅▅
```

from scratch — what a model is, what attention does, why GPUs are suddenly the bottleneck of the AI industry. The last chapter is a hands-on walkthrough of the flags: run this command, see this lesson, run the next one. A bright high-schooler can follow it.

Read either document with `minai.cpp` open alongside. The comments in the source are in the same tone and will not contradict the prose.

Flags at a glance

```
 --blocks=N      stack N transformer layers (1..96, default 1 — 96 matches GPT-3 depth)
--ffn=0|1       include the feed-forward sub-layer (default on)
--causal=0|1    causal attention mask (default off)
--layernorm=0|1 Pre-LN normalization before each sub-layer; makes deep stacks trainable
--random=0|1    train on random sequences + measure held-out accuracy (default off)
--seq_len=N     sequence length 1..32 (attention is O(N^2) in this)
--batch=B       examples averaged per step 1..128 (default 1)
--task=NAME     reverse | sort | shift | mod_sum (default reverse)
--steps=N       training iterations (default 800)
--extra_demos=0|1  run three bonus demos after training (hierarchical weight quantization,
                   speculative decoding, KV cache tiering). Each is a real LLM inference
                   technique, shown in miniature. Adds a few seconds.
```

Try `./minai --random=1 --blocks=2 --batch=16 --layernorm=1 --steps=3000` for a taste of how a real LLM trains in miniature — the model has to generalize, not memorize, and with LN + batch the grokking moment should land well before 3000 steps.

Repo layout

```
 minai.cpp            the program — 19 numbered parts, every line commented
Makefile             one-line clang++ build
TRAINER.md  .pdf     operator's manual
GUIDED_TOUR.md .pdf  textbook + hands-on flag tour
README.md            this file
CLAUDE.md            instructions for the AI assistant that maintains this project
md2pdf.sh            regenerates PDFs from the .md sources
LICENSE              MIT
```

Credit

Damien Boureille wrote the ATTN-11 original in PDP-11 MACRO-11 assembly. Dave Plummer ported it to 2.11BSD `as`. This repo is the C++/Apple-Silicon rewrite with extensive annotation, a textbook companion, and runtime flags for blocks / FFN / causal / random / seq_len / batch / task. The core idea — “tiny transformer, every mechanism present, no magic” — is entirely theirs.